Store asymmetric keys in a key container

09/15/2021

Asymmetric private keys should never be stored verbatim or in plain text on the local computer. If you need to store a private key, use a key container. For more information on key containers, see Understanding machine-level and user-level RSA key containers.

> **Note**
>
> The code in this article applies to Windows and uses features not available in .NET Core 2.2 and earlier versions. For more information, see dotnet/runtime#23391 .

Create an asymmetric key and save it in a key container

1. Create a new instance of a CspParameters class and pass the name that you want to call the key container to the CspParameters.KeyContainerName field.
2. Create a new instance of a class that derives from the AsymmetricAlgorithm class (usually RSACryptoServiceProvider or DSACryptoServiceProvider) and pass the previously created `CspParameters` object to its constructor.

> **Note**
>
> The creation and retrieval of an asymmetric key is one operation. If a key is not already in the container, it's created before being returned.
>
> - RSA.ToXmlString
> - DSA.ToXmlString

Delete the key from the key container

1. Create a new instance of a `CspParameters` class and pass the name that you want to call the key container to the CspParameters.KeyContainerName field.

2. Create a new instance of a class that derives from the AsymmetricAlgorithm class (usually `RSACryptoServiceProvider` or `DSACryptoServiceProvider`) and pass the previously created `CspParameters` object to its constructor.

3. Set the RSACryptoServiceProvider.PersistKeyInCsp or the DSACryptoServiceProvider.PersistKeyInCsp property of the class that derives from `AsymmetricAlgorithm` to `false` (`False` in Visual Basic).

4. Call the `Clear` method of the class that derives from `AsymmetricAlgorithm`. This method releases all resources of the class and clears the key container.

Example

The following example demonstrates how to create an asymmetric key, save it in a key container, retrieve the key at a later time, and delete the key from the container.

Notice that code in the `GenKey_SaveInContainer` method and the `GetKeyFromContainer` method is similar. When you specify a key container name for a CspParameters object and pass it to an AsymmetricAlgorithm object with the PersistKeyInCsp property or PersistKeyInCsp property set to `true`, the behavior is as follows:

- If a key container with the specified name does not exist, then one is created and the key is persisted.
- If a key container with the specified name does exist, then the key in the container is automatically loaded into the current AsymmetricAlgorithm object.

Therefore, the code in the `GenKey_SaveInContainer` method persists the key because it is run first, while the code in the `GetKeyFromContainer` method loads the key because it's run second.

C#

```
using System;
using System.Security.Cryptography;

public class StoreKey
{
    public static void Main()
    {
        try
        {
            // Create a key and save it in a container.
            GenKey_SaveInContainer("MyKeyContainer");
```

```
            // Retrieve the key from the container.
            GetKeyFromContainer("MyKeyContainer");

            // Delete the key from the container.
            DeleteKeyFromContainer("MyKeyContainer");

            // Create a key and save it in a container.
            GenKey_SaveInContainer("MyKeyContainer");

            // Delete the key from the container.
            DeleteKeyFromContainer("MyKeyContainer");
        }
        catch (CryptographicException e)
        {
            Console.WriteLine(e.Message);
        }
    }

    private static void GenKey_SaveInContainer(string containerName)
    {
        // Create the CspParameters object and set the key container
        // name used to store the RSA key pair.
        var parameters = new CspParameters
        {
            KeyContainerName = containerName
        };

        // Create a new instance of RSACryptoServiceProvider that accesses
        // the key container MyKeyContainerName.
        using var rsa = new RSACryptoServiceProvider(parameters);

        // Display the key information to the console.
        Console.WriteLine($"Key added to container: \n  {rsa.ToXmlString(true)}");
    }

    private static void GetKeyFromContainer(string containerName)
    {
        // Create the CspParameters object and set the key container
        // name used to store the RSA key pair.
        var parameters = new CspParameters
        {
            KeyContainerName = containerName
        };

        // Create a new instance of RSACryptoServiceProvider that accesses
        // the key container MyKeyContainerName.
        using var rsa = new RSACryptoServiceProvider(parameters);

        // Display the key information to the console.
        Console.WriteLine($"Key retrieved from container : \n
```

```
{rsa.ToXmlString(true)}");
    }

    private static void DeleteKeyFromContainer(string containerName)
    {
        // Create the CspParameters object and set the key container
        // name used to store the RSA key pair.
        var parameters = new CspParameters
        {
            KeyContainerName = containerName
        };

        // Create a new instance of RSACryptoServiceProvider that accesses
        // the key container.
        using var rsa = new RSACryptoServiceProvider(parameters)
        {
            // Delete the key entry in the container.
            PersistKeyInCsp = false
        };

        // Call Clear to release resources and delete the key from the container.
        rsa.Clear();

        Console.WriteLine("Key deleted.");
    }
}
```

The output is as follows:

Console

```
Key added to container:
<RSAKeyValue> Key Information A</RSAKeyValue>
Key retrieved from container :
<RSAKeyValue> Key Information A</RSAKeyValue>
Key deleted.
Key added to container:
<RSAKeyValue> Key Information B</RSAKeyValue>
Key deleted.
```

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- Generating keys for encryption and decryption

- Encrypting data
- Decrypting data
- ASP.NET Core Data Protection